

Flashcard Application — Project Summary

Final Exam Project · Java & JavaFX · github.com/BeyondM04/FlashcardApp · May 2026

Github link : <https://github.com/BeyondM04/FlashcardApp.git>

INTRODUCTION

The Flashcard Application is a desktop study tool developed in Java using the JavaFX graphical user interface framework as a Final Exam Project. The goal was to design and build a practical, fully functional application that demonstrates core software development concepts including object-oriented design, multi-user session management, persistent data storage, and interactive UI construction — all without relying on any external libraries beyond the standard Java and JavaFX APIs.

The application allows multiple users to register personal accounts, sign in securely, and manage their own private collection of flashcards. Each card stores a question on one side and an answer on the other, replicating the experience of studying with physical index cards while adding the convenience of digital persistence across sessions.

SYSTEM ARCHITECTURE

The project is organized across three Java source files, each with a clearly defined responsibility. The *Main.java* class serves as the JavaFX application entry point, extending the `Application` class and delegating control to `FlashcardApp` on launch. The *Flashcard.java* class is a simple Plain Old Java Object that acts as the data model, storing a question and answer string with standard getters and setters. The *FlashcardApp.java* class is the heart of the application, containing all user interface construction, user session logic, card management operations, and file input/output handling.

At runtime, user accounts are stored in a `HashMap` mapping usernames to passwords, and each user's flashcard collection is maintained in a separate `HashMap` mapping usernames to a list of `Flashcard` objects. All card operations are scoped through a `loggedInUser` session variable, ensuring each user can only view and modify their own data.

FEATURES & FUNCTIONALITY

When the application starts, it loads any previously saved data from disk and presents an authentication screen with Sign In and Sign Up options. The sign-up flow validates that the username is not empty, that the two password fields match, and that the chosen username does not already exist. The sign-in flow checks the entered credentials against the stored values and sets the active session on success.

Once signed in, the user is presented with the main flashcard screen. Cards can be navigated using Previous and Next buttons, which wrap around circularly using modular arithmetic so the deck is always continuous. The Flip button toggles between the question and answer sides, applying a different background color to each side to provide immediate visual feedback about which side is currently displayed. The Shuffle button randomizes the entire deck in place using Java's built-in `Collections.shuffle()` method and resets navigation to the first card.

Users can add new cards through a two-step dialog that collects the question first and then the answer. Cards can be edited with pre-filled dialogs showing the existing content, and deleted after a confirmation prompt that displays the card question to prevent accidental removal. Every write operation — add, edit, delete, and sign-up — immediately triggers a save to disk so no data is lost if the application is closed unexpectedly.

DATA PERSISTENCE

The application stores all user and flashcard data in a plain-text `Properties` file called `data.properties`, written to the application's working directory. This approach was chosen specifically to avoid any external database dependency while still providing reliable persistence. The storage format uses hierarchical dot-notation keys to represent structured data within the flat key-value format of a `Properties` file, as shown below.

```
user.alice = password123 cards.alice.count = 2 cards.alice.0.q = What is Java? cards.alice.0.a = A programming language
cards.alice.1.q = What is JavaFX? cards.alice.1.a = A UI framework
```

The `saveData()` method clears and rewrites the `Properties` object on every change, then persists it using a `FileWriter`. The `loadData()` method reads the file at startup and reconstructs all user accounts and flashcard lists from the stored keys. The file is human-readable and can be inspected or manually edited in any text editor.

Flashcard Application — Project Summary (continued)

Final Exam Project · Java & JavaFX · github.com/BeyondM04/FlashcardApp · May 2026

USER INTERFACE DESIGN

The interface is built entirely in Java code using JavaFX layout containers. The authentication screen measures 400 by 300 pixels and presents only the two main actions — sign in and sign up — keeping the entry point clean and uncluttered. The sign-up and sign-in dialogs are modal windows that block interaction with the main window while open, ensuring the user completes one task at a time. The main flashcard screen measures 540 by 430 pixels and is divided into a top bar showing the logged-in username and a sign-out button, a card counter label, a large card display panel, a navigation row with Previous, Flip, and Next buttons, and an action row with Shuffle, Add, Edit, and Delete buttons.

The card panel is implemented as a StackPane so the label is always centered regardless of text length. Inline CSS is applied directly to JavaFX nodes for all styling, including rounded corners, border colors, and background fills. Buttons use color coding by function to make the interface intuitive at a glance, and all buttons respond to hover events by reducing their opacity slightly to signal interactivity.

ERROR HANDLING & VALIDATION

The application handles user errors at every interaction point. During sign-up, specific inline error messages appear inside the dialog for each type of invalid input — empty fields, mismatched passwords, and duplicate usernames — without closing the dialog so the user can correct their input immediately. During sign-in, an invalid credentials message is shown inline in the same way. When adding or editing a card, blank input silently cancels the operation without modifying the deck. Deleting a card on an empty deck shows an informational alert, and removing a card always clamps the current index to a safe value using `Math.max(0, cards.size() - 1)` to prevent an array index exception. File I/O exceptions in both `saveData()` and `loadData()` are caught and reported via an alert dialog, ensuring the application does not crash silently.

CHALLENGES & SOLUTIONS

The most significant design challenge was representing structured, multi-user data without a relational database. This was solved by generating hierarchical dot-notation keys — such as `cards.alice.0.q` — within a flat Properties file, effectively creating a lightweight structured format using only the standard library. Isolating each user's card deck was achieved by keying all data structures on the username string and routing every card operation through the `loggedInUser` session variable, so there is no risk of one user's data bleeding into another's.

Keeping shared UI labels synchronized across screen reloads presented another challenge. Because the main screen is fully rebuilt on every navigation or shuffle action, the `cardLabel` and `counterLabel` instances are declared as class-level fields rather than local variables, ensuring they persist and are always refreshed by the `updateDisplay()` call at the end of each screen construction. Providing clear visual feedback on the flip action was handled by switching the StackPane's inline CSS background between a warm yellow tone for questions and a light blue tone for answers on every flip toggle.

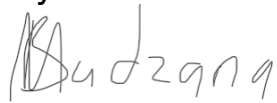
LIMITATIONS & FUTURE WORK

In its current form the application stores passwords as plain text in the data file, which would be unacceptable in a production environment. A real deployment would replace this with a hashing algorithm such as BCrypt. The Properties file format also has limited support for special characters and does not scale well beyond a few thousand cards, so a future version would benefit from replacing it with an embedded SQLite database. Other improvements that were considered but not implemented within the project scope include a quiz mode that tracks correct and incorrect answers and displays a score, card categories or tags to organize cards into multiple named decks, a search function to find cards by keyword, a card flip animation using JavaFX transitions, and an import/export feature to share flashcard sets as CSV or JSON files.

CONCLUSION

The Flashcard Application meets all of its core project objectives. It delivers a working multi-user authentication system, full create-read-update-delete functionality for flashcards, reliable file-based persistence, and a clean, functional graphical interface — built entirely with standard Java and JavaFX and without any external dependencies. The project reinforced important software development principles including separation of concerns through class decomposition, defensive input validation, graceful error handling, and the design of persistent storage schemas. The codebase is structured in a way that makes future enhancements straightforward, and it serves as a solid foundation for a more feature-rich study

Beyond Mudzana- 25120102031

A handwritten signature in black ink, appearing to read 'Budzana'.

Nailejilej Wilbert Mbille - 25120102043

A handwritten signature in black ink, appearing to be a stylized representation of the name 'Nailejilej Wilbert Mbille'.